

Los sistemas de información - Parte 2

1. El rol estratégico de los sistemas y la tecnología de la información (TI)

Un **sistema de información (SI)** es mucho más que solo *software*; es un conjunto de componentes interrelacionados que recolectan, procesan, almacenan y distribuyen información para apoyar la toma de decisiones, la coordinación y el control en una organización.

1.1. Procesos de negocio

La base de toda organización son sus **procesos de negocio**. Estos son conjuntos de actividades lógicamente relacionadas que definen cómo se realizan tareas específicas para producir un producto o servicio. Un sistema de información es eficaz solo si logra soportar, optimizar o transformar estos procesos.

- **Ejemplo de proceso de negocio: “Gestión de un pedido de venta”**
 - I. **Ventas:** un cliente realiza un pedido. El vendedor ingresa el pedido en el sistema.
 - II. **Logística/Almacén:** el sistema notifica al almacén. El personal del almacén prepara los productos.
 - III. **Contabilidad:** el sistema genera la factura correspondiente y la envía al cliente.
 - IV. **Envíos:** el área de logística despacha el pedido.
- Como se puede ver, este proceso involucra a múltiples áreas funcionales. Un sistema de información integrado (como un ERP) es crucial para coordinar estas actividades de manera eficiente.

1.2. Infraestructura de TI

La **tecnología de la información (TI)** es la herramienta que habilita a los sistemas de información modernos. Su infraestructura no es solo *hardware* y *software*, sino un ecosistema que incluye:

- **Hardware:** servidores, computadoras, dispositivos de red.
- **Software:** sistemas operativos, sistemas de gestión de bases de datos (SGBD), aplicaciones de negocio.
- **Almacenamiento de datos:** bases de datos, *data warehouses*.

- **Redes y telecomunicaciones:** Internet, intranets, redes locales (LAN).
- **Recursos humanos:** el personal técnico que administra y opera toda la infraestructura (DBA, administradores de sistemas, desarrolladores).

Las tendencias actuales, como la computación en la nube (*cloud computing*), el crecimiento de las plataformas digitales y el uso de *software* como servicio (SaaS) están redefiniendo constantemente esta infraestructura, haciéndola más flexible y escalable.

2. El ciclo de vida del desarrollo de sistemas (CVDS)

El desarrollo de un sistema de información no es un acto único, sino un proceso estructurado, conocido como el ciclo de vida del desarrollo de sistemas (CVDS). Este ciclo asegura que el proyecto se realice de manera ordenada, controlada y cumpla con los objetivos del negocio. Las fases principales son:

- I. **Investigación preliminar y análisis:** ¿cuál es el problema y qué se necesita?
- II. **Determinación de requerimientos:** recopilar y documentar las necesidades específicas.
- III. **Diseño del sistema:** ¿cómo se resolverá el problema? (creación de modelos).
- IV. **Desarrollo del sistema:** construcción del *software*.
- V. **Prueba del sistema:** verificación de que el sistema funciona correctamente.
- VI. **Implantación:** puesta en marcha del sistema.
- VII. **Mantenimiento y seguimiento:** operación y mejora continua.

A continuación, detallaremos las fases más importantes.

3. Fase de análisis: comprendiendo el problema

Esta es la fase más crítica. Un error en el análisis, casi con seguridad, conducirá al fracaso del proyecto.

3.1. Técnicas de relevamiento de información

Para entender las necesidades, el analista de sistemas debe recopilar información. Esto es vital porque los analistas no conocen los detalles operativos de la organización y los usuarios no siempre saben articular sus necesidades en términos técnicos. Las técnicas de relevamiento sirven como puente de comunicación.

- **Entrevistas:**
 - **No estructurada:** conversación abierta para explorar problemas y oportunidades. Ideal para las primeras etapas.
 - **Estructurada:** se utiliza una lista de preguntas predefinidas. Útil para obtener datos específicos y comparables entre varios entrevistados.
Ejemplo práctico: una entrevista no estructurada con el gerente de ventas, para entender su visión general, seguida de entrevistas estructuradas a los vendedores, para detallar los pasos que siguen al cargar un pedido.
- **Cuestionarios:**
 - Son ideales cuando los usuarios son muchos o están geográficamente dispersos. Permiten el anonimato, lo que puede generar respuestas más sinceras.
 - **Preguntas cerradas:** ofrecen opciones limitadas (sí/no, opción múltiple). Son fáciles de cuantificar.
Ejemplo: “Valore la velocidad del sistema actual: a) Rápida, b) Aceptable, c) Lenta”.
 - **Preguntas abiertas:** permiten respuestas libres. Ofrecen información cualitativa rica, pero son más difíciles de analizar.
Ejemplo: “Describa el mayor desafío que enfrenta al utilizar el sistema actual”.
- **Observación:**
 - El analista observa a los usuarios mientras realizan sus tareas, para entender el flujo de trabajo real, que a menudo difiere de los procedimientos escritos. Puede ser **directa** (planificada) o **indirecta** (espontánea).
 - **Consideración teórica (efecto Hawthorne):** las personas pueden cambiar su comportamiento, cuando saben que están siendo observadas. El analista debe ser consciente de esto.
- **Revisión de registros:**
 - Análisis de documentación existente, como manuales de procedimiento, facturas, reportes y formularios. Esto proporciona una visión formal de cómo debería funcionar la organización.

3.2. Creación de modelos: lógico y físico

Una vez recopilada la información, el analista debe modelar la solución. Es fundamental distinguir entre el modelo lógico y el físico.

- **Modelo lógico**

- **Objetivo:** representar la solución conceptualmente, describiendo **qué** hará el sistema, sin importar **cómo** se implementará tecnológicamente. Es un lenguaje común entre el usuario y el analista.
- **Define:**
 - **Flujos de información:** de dónde viene y a dónde va.
 - **Procesos:** qué transformaciones sufre la información.
 - **Almacenes de datos:** qué información se debe guardar.
- **Analogía:** es el plano arquitectónico de una casa. Muestra las habitaciones, sus tamaños y cómo se conectan, pero no especifica el tipo de ladrillos o cables a usar.

- **Modelo físico**

- **Objetivo:** traducir el modelo lógico en especificaciones técnicas detalladas. Describe **cómo** se construirá el sistema, considerando las limitaciones de *hardware*, *software* y tecnología.
- **Define:**
 - **Diseño de la interfaz:** cómo serán las pantallas y los reportes.
 - **Diseño de la base de datos:** las tablas, campos, tipos de datos y relaciones.
 - **Especificaciones de procesos:** la lógica de programación, los algoritmos.
- **Analogía:** son los planos de construcción e ingeniería. Detallan el tipo de cimientos, el calibre del cableado eléctrico, el material de las tuberías, etc.

4. Fase de diseño: construyendo la solución

En esta fase, los modelos se refinan para crear especificaciones completas. Un área clave es el diseño de la base de datos.

4.1. Almacenamiento y normalización de datos

La normalización es un proceso formal para diseñar bases de datos relacionales, con el objetivo de reducir la redundancia de datos y mejorar su integridad.

- **Ejemplo práctico de normalización:** imaginemos una tabla simple para registrar pedidos:

PedidoID	Fecha	CienteID	Nombre cliente	ProductoID	Nombre producto	Cantidad	Precio unitario
101	01/10/25	C01	Juan Pérez	P55	Teclado	2	1500
101	01/10/25	C01	Juan Pérez	P62	Mouse	2	800
102	02/10/25	C05	Ana López	P55	Teclado	1	1500

Esta tabla tiene problemas: si Juan Pérez cambia su nombre, hay que actualizarlo en múltiples filas. Si borramos el pedido 102, perdemos la información de que el producto P55 es un “Teclado”.

- **Primera forma normal (1FN):** la tabla ya es bidimensional y no tiene grupos repetitivos, por lo que cumple 1FN.
- **Segunda forma normal (2FN):** elimina dependencias parciales. Los datos que no dependen de la clave completa se mueven a otra tabla. La clave aquí es (PedidoID, ProductoID).
 - *Problema:* Fecha, CienteID y NombreCliente dependen solo de PedidoID, no de la clave completa.
 - *Solución:* dividimos las tablas. **Tabla pedidos:** | PedidoID (PK) | Fecha | CienteID | NombreCliente | | :--- | :--- | :--- | :--- | **Tabla detalle pedido:** | PedidoID (FK) | ProductoID (FK) | Cantidad |
- **Tercera forma normal (3FN):** elimina dependencias transitivas.
 - *Problema:* en la nueva tabla Pedidos, NombreCliente depende de CienteID, que, a su vez, depende de PedidoID. Esto es una dependencia transitiva.

- *Solución:* volvemos a dividir. **Tabla clientes:** | ClienteID (PK) | NombreCliente | | :--- | :--- | **Tabla productos:** | ProductoID (PK) | NombreProducto | PrecioUnitario | **Tabla pedidos:** | PedidoID (PK) | Fecha | ClienteID (FK) | **Tabla detalle pedido:** | PedidoID (FK) | ProductoID (FK) | Cantidad |
- Ahora, los datos están organizados, sin redundancia, y son más fáciles de mantener.

5. Fases de prueba e implementación

5.1. Estrategias de prueba

Probar es fundamental para garantizar la calidad.

- **Prueba de código (*White box*):** el programador examina la lógica interna del código, para asegurarse de que cada rama y bucle funcione como se espera.
- **Prueba de especificación (*Black box*):** el tester verifica los resultados del sistema, sin conocer el código interno. Se enfoca en si el sistema cumple con los requerimientos funcionales.
- **Pruebas especiales:**
 - **Prueba de carga máxima:** ¿soporta el sistema a 500 usuarios conectados a la vez?
 - **Prueba de recuperación:** si el servidor se apaga, ¿se pueden restaurar los datos desde el *backup*?
 - **Prueba de factores humanos (Usabilidad):** ¿es el sistema intuitivo y fácil de usar para un nuevo empleado?

5.2. Puesta en marcha (Implementación)

Existen diferentes estrategias para lanzar el nuevo sistema:

- **En paralelo:** el sistema viejo y el nuevo funcionan a la vez, durante un período. Es la más segura, pero la más costosa.
- **Gradual (o por fases):** se implementa el nuevo sistema, por módulos o departamentos.

- **Piloto:** se implementa el sistema completo en una pequeña parte de la organización. Si funciona, se expande al resto.
- **Abrupta (o *Big bang*):** se apaga el sistema viejo y se enciende el nuevo en una fecha determinada. Es la más riesgosa pero la más rápida.

6. Mantenimiento y entornos de sistemas

6.1. Tipos de mantenimiento

Una vez implementado, el sistema entra en su fase más larga: el mantenimiento. Representa la mayor parte del costo total de propiedad de un sistema.

- **Correctivo (20 %):** arreglar errores y fallos ("*bugs*") que no se detectaron en las pruebas.
- **Adaptativo (20 %):** modificar el sistema para que se adapte a cambios en el entorno, como un nuevo sistema operativo o cambios en la legislación.
- **Perfectivo (60 %):** mejorar el sistema agregando nuevas funcionalidades solicitadas por los usuarios u optimizando su rendimiento.

6.2. Entornos de sistemas

Para un desarrollo profesional y seguro, es indispensable trabajar con múltiples entornos:

- **Desarrollo (DEV):** ambiente local de los programadores, donde escriben y prueban su código. Es un entorno inestable.
- **Calidad (QA - *Quality assurance*):** una réplica del entorno real, donde los testers realizan pruebas formales para validar que los cambios funcionan y no rompen nada existente.
- **Producción (PROD):** es el entorno **real**, donde los usuarios finales trabajan. Es el sistema "en vivo" y debe ser estable y estar protegido.

El pasaje de código de DEV a QA y luego a PROD sigue un proceso controlado para minimizar los riesgos.